

컴퓨터네트워크 소켓프로그래밍 Server, Client 프로그램 작성 과제

임시원 (20243106)

국민대학교 소프트웨어학부



02 동작 환경

공유기 포트 포워딩을 통해
외부에서 공유기를 통해
서버로 접속할 수 있도록
설계하였습니다.

메뉴탐색기

기본 설정

시스템 요약 정보

인터넷 설정 정보

무선 설정/보안

펌웨어 업그레이드

Easy Mesh

고급 설정

네트워크 관리

인터넷 설정 정보

내부 네트워크 설정

DHCP 서버 설정

무선랜 관리

NAT/라우터 관리

포트포워드 설정

고급 NAT 설정

라우팅 테이블 관리

보안 기능

특수기능

트래픽 관리

시스템 관리

UPN/서비스 관리

ipTIME A9004M-X2

다시 저장 도움

포트포워드 설정

사용자정의 규칙보기

순위	사용자 규칙	내부 IP	외부 포트	내부 포트	삭제
1	rpi	192.168.0.151	TCPUDP(1102~1102)	TCPUDP(80~80)	☐
2	rpiftp	192.168.0.151	TCPUDP(20~21)	TCPUDP(20~21)	☐
3	8080	192.168.0.237	TCPUDP(8080~8080)	TCPUDP(8080~8080)	☐
+ 새규칙 추가					

규칙이름

8080

포트포워드 사용자정의

☐ 규칙 비활성화

순위 3

내부 IP주소

192.168.0.237

☐ 현재 접속된 IP 주소

프로토콜

TCP/UDP

외부 포트

8080 ~ 8080

내부 포트

8080 ~ 8080

PC<-규칙저장

PC->규칙복원

파일 선택

선택된 파일 없음

새규칙

수정

취소

02 Method-응답 HTTP 응답 코드 case 5개

HTTP 응답 코드	발생 조건
GET 200 OK	index 경로 or /경로에 GET 요청을 하였을 때
GET 404 Not Found	nf 경로로 GET 요청을 하였을 때
GET 500 Internal Server Error	ise 경로에 GET 요청을 하였을 때
POST 200 OK	index에 POST 요청을 하였을 때
POST 404 Not Found	nf 경로에 POST 요청을 하였을 때
400 Bad Request	GET, POST 이외의 메서드로 요청 하였을 때

Server Code

02 코드 구조 (헤더)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
```

stdio.h는 출력printf, perror위해 포함 하였고

stdlib.h는 프로그램 exit 같은 제어나, 메모리 할당할 때 사용되기 때문에 포함하였습니다

string.h는 문자열 비교(strcmp), 문자열 길이(strlen)를 재는 데 필요하여 포함하였습니다

unistd.h는 파일 조작(close, read, write)같은 시스템 콜을 하기 위해 포함시켰습니다

arpa/inet.h는 TCP/IP 소켓 통신을 할 때 필요한 함수들(소켓 생성, 주소 바인딩 등)을 위해 포함하였습니다

02 서버 → 클라이언트 응답 보내는 함수

```
10 void send_response(int client_fd, int code, const char *status, const char *body)
11 {
12     char response[4096];
13
14
15     snprintf(response, sizeof(response),
16              "HTTP/1.1 %d %s\r\n"
17              "Content-Type: text/html\r\n"
18              "Content-Length: %lu\r\n"
19              "Connection: close\r\n"
20              "\r\n"
21              "%s",
22              code, status, strlen(body), body);
23
24
25     write(client_fd, response, strlen(response));
26 }
```

코드 설명은 다음 페이지에 작성하였습니다

02 서버 → 클라이언트 응답 보내는 함수

```
10 void send_response(int client_fd, int code, const char *status, const char *body)
11 {
12     char response[4096];
13
14     snprintf(response, sizeof(response),
15              "HTTP/1.1 %d %s\r\n"
16              "Content-Type: text/html\r\n"
17              "Content-Length: %lu\r\n"
18              "Connection: close\r\n"
19              "\r\n"
20              "%s",
21              code, status, strlen(body), body);
22
23     write(client_fd, response, strlen(response));
24
25 }
26
```

이 함수는 클라이언트에게 HTTP 응답을 보내기 위해 작성하였습니다.

먼저, 4096바이트 크기의 response 버퍼를 선언하여, HTTP 응답 메시지를 임시로 저장할 공간을 마련하였습니다.

그 다음 snprintf 함수를 이용하여, HTTP 응답 포맷에 맞게 응답 메시지를 조립하였습니다.

snprintf 함수는 버퍼의 크기를 넘지 않도록 안전하게 문자열을 조합하는 함수로, 이 코드에서는 response 버퍼에 다음과 같은 내용을 작성하였습니다.

첫 줄에는 "HTTP/1.1 %d %s\r\n" 포맷을 사용하여, HTTP 버전과 상태 코드, 상태 메시지를 삽입하였습니다.

이후 "Content-Type: text/html\r\n"을 추가하여 본문의 타입이 HTML임을 명시하였고,

"Content-Length: %lu\r\n"을 통해 본문의 길이를 나타내는 Content-Length 헤더를 작성하였습니다.

그 다음 "Connection: close\r\n"을 추가하여 서버가 응답 이후 연결을 종료할 것임을 명시하였습니다.

헤더 작성이 완료된 후에는 \r\n을 추가하여 헤더와 본문 사이를 구분하였으며, 마지막으로 %s 자리에 HTML 본문 데이터인 body 문자열을 삽입하였습니다.

조립이 완료된 응답 메시지는 write 함수를 통해 클라이언트 소켓 파일 디스크립터(client_fd)에 전송하였습니다.

전송 시에는 조립한 response 문자열의 길이만큼 전송하였고, 이를 통해 클라이언트는 HTTP 응답을 정상적으로 수신할 수 있도록 하였습니다.

이 함수는 HTTP/1.1 규격을 준수하여, 상태 코드, Content-Type, Content-Length, 연결 종료 명령을 모두 포함하고 있으며,

특히 Content-Length를 명확히 지정함으로써 클라이언트가 본문을 정확하게 읽을 수 있도록 처리하였습니다.

02 main() 함수 설명 (TCP 소켓, 생성 설정)

```
28 int main()
29 {
30     int server_fd, client_fd;
31     struct sockaddr_in addr;
32     socklen_t addrlen = sizeof(addr);
33     char buffer[4096];
34     server_fd = socket(AF_INET, SOCK_STREAM, 0);
35     if (server_fd == -1)
36     {
37         perror("socket failed");
38         exit(EXIT_FAILURE);
39     }
40     int opt = 1;
41     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
42
43
44     addr.sin_family = AF_INET;
45     addr.sin_addr.s_addr = INADDR_ANY;
46     addr.sin_port = htons(PORT);
47
48
49     if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
50     {
51         perror("bind failed");
52         close(server_fd);
53         exit(EXIT_FAILURE);
54     }
55
56
57     if (listen(server_fd, 10) < 0)
58     {
59         perror("listen failed");
60         close(server_fd);
61         exit(EXIT_FAILURE);
62     }
```

코드 설명은 다음 페이지에 작성하였습니다

02 main() 함수 설명 (TCP 소켓, 생성 설정)

main() 함수에서는, TCP 소켓을 생성하고 설정하여 클라이언트 접속을 받을 준비를 하는 역할을 수행하였습니다.

먼저 서버 소켓 디스크립터(server_fd)와 클라이언트 소켓 디스크립터(client_fd)를 선언하였고, 서버의 IP 및 포트 정보를 저장할 sockaddr_in 구조체와 주소 길이를 저장할 socklen_t 변수를 선언하였습니다.

또한 클라이언트 요청을 저장할 버퍼(buffer)를 4096바이트 크기로 선언하여, 이후 요청을 받을 준비를 하였습니다.

서버 소켓은 socket(AF_INET, SOCK_STREAM, 0) 함수를 호출하여 생성하였습니다.

여기서 AF_INET은 IPv4 인터넷 프로토콜을 의미하고, SOCK_STREAM은 TCP 스트림 소켓을 의미합니다.

소켓 생성이 실패한 경우 perror를 통해 에러 메시지를 출력하고 exit(EXIT_FAILURE)를 호출하여 프로그램을 종료하도록 작성하였습니다.

소켓 재사용을 허용하기 위해 setsockopt 함수를 호출하여 SO_REUSEADDR 옵션을 설정하였습니다.

이 설정은 서버를 재시작할 때 포트가 TIME_WAIT 상태에 있어도 재사용할 수 있도록 해주며, 서버 운영의 편의성을 높이기 위해 추가하였습니다.

서버 주소 구조체(addr)의 각 필드를 초기화하였습니다.

sin_family 필드에는 AF_INET을 설정하여 IPv4 주소 체계를 사용하도록 하였고,

sin_addr.s_addr에는 INADDR_ANY를 설정하여 서버가 모든 IP 주소(로컬 호스트 포함)로부터 접속을 받을 수 있도록 설정하였습니다.

또한 sin_port 필드에는 사용할 포트 번호를 htons(PORT)를 통해 네트워크 바이트 순서로 변환하여 저장하였습니다.

소켓과 주소를 바인딩하기 위해 bind 함수를 호출하였습니다.

바인딩이 실패한 경우 에러 메시지를 출력하고 서버 소켓을 닫은 후 프로그램을 종료하도록 처리하였습니다.

마지막으로 클라이언트 접속을 대기 상태로 만들기 위해 listen 함수를 호출하였습니다.

백로그(대기 큐 크기)는 10으로 설정하여 동시에 최대 10개의 클라이언트 연결 요청을 대기할 수 있도록 하였습니다.

listen 호출이 실패한 경우에도 에러 메시지를 출력하고 서버 소켓을 닫은 후 프로그램을 종료하도록 작성하였습니다.

02 바인드, listen 부분

```
49     if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
50     {
51         perror("bind failed");
52         close(server_fd);
53         exit(EXIT_FAILURE);
54     }
55
56
57     if (listen(server_fd, 10) < 0)
58     {
59         perror("listen failed");
60         close(server_fd);
61         exit(EXIT_FAILURE);
62     }
```

코드 설명은 다음 페이지에 작성하였습니다

02 바인드, listen 부분

이 부분 코드는 서버 소켓을 생성한 이후, 소켓에 서버 주소를 바인딩하고 클라이언트의 연결 요청을 받을 수 있도록 리슨 상태로 만드는 과정을 처리하였습니다.

먼저 `bind()` 함수를 호출하여, 서버 소켓(`server_fd`)과 서버 주소 구조체(`addr`)를 바인딩하였습니다.

`bind()` 호출 시, 만약 리턴값이 0보다 작다면 바인딩에 실패한 것으로 간주하고 `perror("bind failed")`를 호출하여 에러 메시지를 출력하였습니다. 그 후, 서버 소켓을 `close(server_fd)`를 통해 정상적으로 닫아주었으며, 프로그램을 `exit(EXIT_FAILURE)`를 호출하여 비정상 종료시키도록 처리하였습니다.

다음으로 `listen()` 함수를 호출하여 서버 소켓을 클라이언트 요청을 수신 대기할 수 있는 상태로 전환하였습니다.

`listen()` 함수에는 백로그 크기(대기할 수 있는 클라이언트 요청 수)로 10을 설정하였습니다.

만약 `listen()` 호출이 실패할 경우에도 `perror("listen failed")`를 이용하여 에러 메시지를 출력하였고, 서버 소켓을 `close(server_fd)`로 닫은 후, `exit(EXIT_FAILURE)`로 프로그램을 비정상 종료하도록 작성하였습니다.

이렇게 서버는 소켓 생성 후 주소 바인딩 및 연결 대기 준비를 모두 마친 상태가 되고

각 단계마다 에러 발생 시 적절히 리소스를 해제하고 프로그램을 종료하는 안정적인 처리를 구현하였습니다.

02 요청 데이터 처리 부분

```
while (1)
{

    client_fd = accept(server_fd, (struct sockaddr *)&addr, &addrlen);
    if (client_fd < 0)
    {
        perror("accept failed");
        continue;
    }

    int bytes_read = read(client_fd, buffer, sizeof(buffer) - 1);
    if (bytes_read > 0)
    {
        buffer[bytes_read] = '\0';
        printf("Received request:\n%s\n", buffer);

        char method[8], path[1024];
        sscanf(buffer, "%s %s", method, path);
    }
}
```

코드 설명은 다음 페이지에 작성하였습니다

02 GET 요청 데이터 처리 부분

먼저, while (1) 무한 루프를 통해 서버가 지속적으로 클라이언트 요청을 대기하도록 하였습니다.

루프 내부에서는 accept() 함수를 호출하여 클라이언트 연결 요청을 수락하였습니다.

accept() 호출 시, 새로운 클라이언트와의 통신을 위한 소켓 디스크립터(client_fd)를 반환받습니다.

만약 accept() 호출이 실패하여 반환값이 0보다 작을 경우, perror("accept failed")를 호출하여 에러 메시지를 출력하고, continue를 사용하여 현재 루프를 건너뛰고 다음 클라이언트 연결을 기다리도록 하였습니다.

클라이언트 연결이 정상적으로 이루어진 경우, read() 함수를 호출하여 클라이언트로부터 데이터를 읽어왔습니다.

read() 함수는 client_fd로부터 최대 sizeof(buffer) - 1 만큼의 바이트를 읽어 buffer에 저장하였습니다.

읽은 데이터의 크기는 bytes_read 변수에 저장되며, 읽은 데이터가 있을 경우(bytes_read > 0),

buffer[bytes_read] = '\0';를 통해 읽은 데이터 뒤에 문자열 종료 문자(NULL)를 삽입하여 안전한 문자열로 만들었습니다.

이후 printf() 함수를 통해 클라이언트로부터 수신한 요청 전체를 출력하였습니다.

이 과정을 통해 서버는 클라이언트가 보낸 HTTP 요청 전체를 콘솔에 출력할 수 있습니다.

마지막으로, 요청 메시지에서 HTTP 메서드와 경로를 분리하기 위해 char method[8], path[1024];를 선언하였고

sscanf(buffer, "%s %s", method, path);를 호출하여 buffer 문자열에서 첫 번째 단어(메서드)와 두 번째 단어(경로)를 추출하였습니다.

서버는 클라이언트가 요청한 HTTP 메서드(GET, POST 등)와 요청 경로(/index, /nf 등)를 분리하여 이후 처리할 수 있도록 준비를 완료하였습니다.

02 GET 요청 데이터 처리 부분

```
if (strcmp(method, "GET") == 0)
{
    if (strcmp(path, "/index") == 0 || strcmp(path, "/") == 0)
    {
        send_response(client_fd, 200, "OK",
            "<html><body><h1>200 OK (GET)</h1><p>Hello siwon</p></body></html>");
    }
    else if (strcmp(path, "/nf") == 0)
    {
        send_response(client_fd, 404, "Not Found",
            "<html><body><h1>404 Not Found (GET)</h1><p>Page not found.</p></body></html>");
    }
    else if (strcmp(path, "/ise") == 0)
    {
        send_response(client_fd, 500, "Internal Server Error",
            "<html><body><h1>500 Internal Server Error (GET)</h1><p>Server error occurred.</p></body></html>");
    }
}
```

코드 설명은 다음 페이지에 작성하였습니다

02 요청 데이터 처리 부분

이 코드는 클라이언트로부터 수신한 HTTP 요청 중, 요청 메서드가 GET일 경우에 대해 처리하는 로직을 작성하였습니다.

먼저 `strcmp(method, "GET") == 0` 조건문을 사용하여, 요청 메서드가 GET인지 비교하였습니다.

요청 메서드가 "GET"임을 확인한 후, 요청 경로에 따라 각각 다른 응답을 전송하도록 분기 처리하였습니다.

첫 번째로, 요청 경로가 `/index` 또는 `/`인 경우를 처리하였습니다.

`strcmp(path, "/index") == 0 || strcmp(path, "/") == 0` 조건을 만족하면,

`send_response()` 함수를 호출하여 200 OK 응답과 함께 간단한 HTML 본문(`<h1>200 OK (GET)</h1><p>Hello siwon</p>`)을 클라이언트에 전송하였습니다.

두 번째로, 요청 경로가 `/nf`인 경우를 처리하였습니다.

`strcmp(path, "/nf") == 0` 조건을 만족할 때, `send_response()` 함수를 호출하여 404 Not Found 응답과 함께 "Page not found"라는 내용을 포함한 HTML 본문을 클라이언트로 전송하였습니다.

세 번째로, 요청 경로가 `/ise`인 경우를 처리하였습니다.

`strcmp(path, "/ise") == 0` 조건을 만족할 때, `send_response()` 함수를 호출하여 500 Internal Server Error 응답과 함께 "Server error occurred"라는 내용을 포함한 HTML 본문을 클라이언트에 전송하였습니다.

02 POST 요청 데이터 처리 부분

```
else if (strcmp(method, "POST") == 0)
{
    if (strcmp(path, "/index") == 0 || strcmp(path, "/") == 0)
    {
        send_response(client_fd, 200, "OK",
            "<html><body><h1>200 OK (POST)</h1><p>POST request successful</p></body></html>");
    }
    else if (strcmp(path, "/nf") == 0)
    {
        send_response(client_fd, 404, "Not Found",
            "<html><body><h1>404 Not Found (POST)</h1><p>Page not found.</p></body></html>");
    }
}
else
{
    send_response(client_fd, 400, "Bad Request",
        "<html><body><h1>400 Bad Request</h1><p>Unsupported method.</p></body></html>");
}
```

코드 설명은 다음 페이지에 작성하였습니다

02 POST 요청 데이터 처리 부분

먼저 `strcmp(method, "POST") == 0` 조건문을 통해, 클라이언트의 요청 메서드가 "POST"인지 확인하였습니다.

요청 메서드가 "POST"일 경우, 요청 경로에 따라 각각 다른 응답을 전송하는 분기 처리를 수행하였습니다.

첫 번째로, 요청 경로가 `/index` 또는 `/`인 경우를 처리하였습니다.

`strcmp(path, "/index") == 0 || strcmp(path, "/") == 0` 조건을 만족하면,

`send_response()` 함수를 호출하여 200 OK 응답과 함

"POST request successful"이라는 본문을 포함하는

두 번째로, 요청 경로가 `/nf`인 경우를 처리하였습니다.

`strcmp(path, "/nf") == 0` 조건을 만족할 경우,

`send_response()` 함수를 호출하여 404 Not Found 응답과 함

"Page not found"라는 본문을 포함하는 HTML 페이지를 클라이언트에 전송하였습니다.

요청 메서드가 "GET"이나 "POST"가 아닌 경우에는,

`else` 블록을 통해 기본적으로 400 Bad Request 응답을 전송하도록 처리하였습니다.

이 경우 `send_response()` 함수를 호출하여, "Unsupported method"라는 본문을 포함하는 HTML 페이지를 클라이언트에 전송하였습니다.

```
133     close(client_fd);  
134 }  
135  
136     close(server_fd);  
137     return 0;  
138 }
```

02 연결 종료시 소켓 정리

```
133         close(client_fd);
134     }
135
136     close(server_fd);
137     return 0;
138 }
```

먼저, 클라이언트와 통신이 끝난 후, `close(client_fd);`를 호출하여 클라이언트 소켓 파일 디스크립터를 명시적으로 닫아주었습니다.

이 과정을 통해 클라이언트와의 연결 리소스를 해제하고, 시스템 리소스 누수를 방지하였습니다.

`while (1)` 루프가 끝난 후, 즉 서버가 종료되기 직전에는 `close(server_fd);`를 호출하여 서버 소켓 파일 디스크립터를 닫아주었습니다.

서버 소켓을 닫음으로써 서버가 더 이상 클라이언트의 연결 요청을 수락할 수 없게 되며, 운영 체제에 할당된 포트를 반납하여 다른 프로세스가 사용할 수 있도록 처리하였습니다.

마지막으로 `return 0;`을 통해 프로그램을 정상 종료하였습니다.

Client Code

02

헤더와 기본 설정

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define SERVER_IP "127.0.0.1"
8 #define SERVER_PORT 8080
9 #define BUFFER_SIZE 4096
```

이 코드는 클라이언트 프로그램을 작성하기 위해 필요한 기본 설정을 포함하였습니다. 먼저 `<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<unistd.h>`, `<arpa/inet.h>` 헤더를 추가하여 입출력, 문자열 처리, 시스템 호출, 네트워크 소켓 프로그래밍에 필요한 기능을 사용할 수 있도록 하였습니다. 그 다음 `#define`을 이용하여 서버의 IP 주소를 "127.0.0.1"로, 포트 번호를 8080으로, 통신에 사용할 버퍼 크기를 4096바이트로 정의하였습니다.

02 Send Request 함수

```
11 void send_request(const char *request)
12 {
13     int sock;
14     struct sockaddr_in server_addr;
15     char buffer[BUFFER_SIZE];
16     int bytes_read;
17
18     sock = socket(AF_INET, SOCK_STREAM, 0);
19     if (sock == -1)
20     {
21         perror("socket failed");
22         exit(EXIT_FAILURE);
23     }
24
25     server_addr.sin_family = AF_INET;
26     server_addr.sin_port = htons(SERVER_PORT);
27     if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0)
28     {
29         perror("inet_pton failed");
30         close(sock);
31         exit(EXIT_FAILURE);
32     }
33
34     if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0)
35     {
36         perror("connect failed");
37         close(sock);
38         exit(EXIT_FAILURE);
39     }
40
41     send(sock, request, strlen(request), 0);
42
43     printf("\n--- 요청 ---\n%s", request);
44     printf("\n--- 응답 ---\n");
45
46     while ((bytes_read = read(sock, buffer, sizeof(buffer) - 1)) > 0)
47     {
48         buffer[bytes_read] = '\0';
49         printf("%s", buffer);
50     }
51     printf("\n-----\n");
52
53     close(sock);
54 }
```

이 코드는 서버에 요청을 보내고 응답을 받기 위한 send_request 함수를 작성하였습니다.

먼저 소켓을 생성하고(socket()), 생성 실패 시 에러를 출력하고 프로그램을 종료하였습니다.

서버 주소 구조체를 설정하고, IP 주소를 변환하기 위해 inet_pton 함수를 호출하였으며, 변환에 실패할 경우 소켓을 닫고 종료하였습니다.

이후 connect 함수를 통해 서버에 연결을 시도하였고, 연결 실패 시에도 에러를 출력하고 소켓을 닫은 후 프로그램을 종료하였습니다.

서버 연결에 성공한 경우, 전달받은 request 문자열을 send 함수를 이용해 서버로 전송하였습니다.

요청을 전송한 후, 서버로부터 응답을 받기 위해 read 함수를 반복적으로 호출하여 버퍼에 데이터를 읽어들이었으며, 읽은 내용을 출력하였습니다.

모든 응답을 받은 뒤에는 소켓을 닫고 함수를 종료하였습니다.

02 Http Request Test

```
56 int main()  
57 {  
58     const char *get_index =  
59         "GET /index HTTP/1.1\r\n"  
60         "Host: localhost:8080\r\n"  
61         "Connection: close\r\n"  
62         "\r\n";  
63  
64     const char *get_root =  
65         "GET / HTTP/1.1\r\n"  
66         "Host: localhost:8080\r\n"  
67         "Connection: close\r\n"  
68         "\r\n";  
69  
70     const char *get_nf =  
71         "GET /nf HTTP/1.1\r\n"  
72         "Host: localhost:8080\r\n"  
73         "Connection: close\r\n"  
74         "\r\n";  
75  
76     const char *get_ise =  
77         "GET /ise HTTP/1.1\r\n"  
78         "Host: localhost:8080\r\n"  
79         "Connection: close\r\n"  
80         "\r\n";
```

```
82     const char *post_index =  
83         "POST /index HTTP/1.1\r\n"  
84         "Host: localhost:8080\r\n"  
85         "Content-Type: application/x-www-form-urlencoded\r\n"  
86         "Content-Length: 9\r\n"  
87         "Connection: close\r\n"  
88         "\r\n"  
89         "data=test";  
90  
91     const char *post_nf =  
92         "POST /nf HTTP/1.1\r\n"  
93         "Host: localhost:8080\r\n"  
94         "Content-Type: application/x-www-form-urlencoded\r\n"  
95         "Content-Length: 9\r\n"  
96         "Connection: close\r\n"  
97         "\r\n"  
98         "data=test";  
99  
100    const char *put_test =  
101        "PUT /index HTTP/1.1\r\n"  
102        "Host: localhost:8080\r\n"  
103        "Content-Length: 0\r\n"  
104        "Connection: close\r\n"  
105        "\r\n";  
106  
107    send_request(get_index);  
108    send_request(get_root);  
109    send_request(get_nf);  
110    send_request(get_ise);  
111    send_request(post_index);  
112    send_request(post_nf);  
113    send_request(put_test);  
114  
115    return 0;  
116 }
```

코드 설명은 다음 페이지에 작성하였습니다

02 Http Request Test

이 코드는 서버에 다양한 HTTP 요청을 보내기 위해 요청 메시지를 미리 정의하고, 이를 전송하는 과정을 작성하였습니다. 먼저 GET, POST, PUT 요청에 사용할 요청 문자열을 각각 `const char*` 형태로 선언하였습니다. `/index`, `/`, `/nf`, `/ise` 경로에 대해 GET 요청을 작성하였고, `/index`와 `/nf` 경로에 대해 POST 요청을 작성하였습니다. 또한 `/index` 경로에 대해 PUT 요청을 작성하여, 다양한 HTTP 메서드 상황을 테스트할 수 있도록 준비하였습니다. 작성된 요청 문자열들은 `send_request()` 함수를 호출하여 차례대로 서버로 전송하였습니다. `send_request()` 호출을 통해 각각의 요청을 서버에 보내고, 그에 대한 응답을 받아 출력하는 과정을 수행하였습니다. 마지막으로 `return 0;`을 통해 프로그램을 정상 종료하였습니다.

02 Http Request Test 결과

```
--- 요청 ---
GET /index HTTP/1.1
Host: localhost:8080
Connection: close
```

```
--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 65
Connection: close
```

```
<html><body><h1>200 OK (GET)</h1><p>Hello siwon</p></body></html>
```

```
--- 요청 ---
GET / HTTP/1.1
Host: localhost:8080
Connection: close
```

```
--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 65
Connection: close
```

```
<html><body><h1>200 OK (GET)</h1><p>Hello siwon</p></body></html>
```

```
--- 요청 ---
GET /nf HTTP/1.1
Host: localhost:8080
Connection: close
```

```
--- 응답 ---
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 76
Connection: close
```

```
<html><body><h1>404 Not Found (GET)</h1><p>Page not found.</p></body></html>
```

```
--- 요청 ---
GET /ise HTTP/1.1
Host: localhost:8080
Connection: close
```

```
--- 응답 ---
HTTP/1.1 500 Internal Server Error
Content-Type: text/html
Content-Length: 95
Connection: close
```

```
<html><body><h1>500 Internal Server Error (GET)</h1><p>Server error occurred.</p></body></html>
```

```
--- 요청 ---
POST /index HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 9
Connection: close
```

```
data=test
```

```
--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 78
Connection: close
```

```
<html><body><h1>200 OK (POST)</h1><p>POST request successful</p></body></html>
```

```
--- 요청 ---
POST /nf HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 9
Connection: close
```

```
data=test
```

```
--- 응답 ---
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 77
Connection: close
```

```
<html><body><h1>404 Not Found (POST)</h1><p>Page not found.</p></body></html>
```

```
--- 요청 ---
PUT /index HTTP/1.1
Host: localhost:8080
Content-Length: 0
Connection: close
```

```
--- 응답 ---
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 76
Connection: close
```

02 Http Request Test 결과

```
--- 요청 ---
GET /index HTTP/1.1
Host: localhost:8080
Connection: close

--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 65
Connection: close

<html><body><h1>200 OK (GET)</h1><p>Hello siwon</p></body></html>
-----

--- 요청 ---
GET / HTTP/1.1
Host: localhost:8080
Connection: close

--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 65
Connection: close

<html><body><h1>200 OK (GET)</h1><p>Hello siwon</p></body></html>
-----

--- 요청 ---
GET /nf HTTP/1.1
Host: localhost:8080
Connection: close

--- 응답 ---
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 76
Connection: close

<html><body><h1>404 Not Found (GET)</h1><p>Page not found.</p></body></html>
-----
```

```
--- 요청 ---
GET /ise HTTP/1.1
Host: localhost:8080
Connection: close

--- 응답 ---
HTTP/1.1 500 Internal Server Error
Content-Type: text/html
Content-Length: 95
Connection: close

<html><body><h1>500 Internal Server Error (GET)</h1><p>Server error occurred.</p></body></html>
-----

--- 요청 ---
POST /index HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 9
Connection: close

data=test

--- 응답 ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 78
Connection: close

<html><body><h1>200 OK (POST)</h1><p>POST request successful</p></body></html>
-----

--- 요청 ---
POST /nf HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 9
Connection: close

data=test

--- 응답 ---
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 77
Connection: close

<html><body><h1>404 Not Found (POST)</h1><p>Page not found.</p></body></html>
-----

--- 요청 ---
PUT /index HTTP/1.1
Host: localhost:8080
Content-Length: 0
Connection: close

--- 응답 ---
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 76
Connection: close
```

결과를 통해 잘 작동됨을 확인하였습니다